

Building with Local AI Agents vs Anthropic Claude Managed Agents

A Technical Comparison of Agent Orchestration Architectures

*Who controls the execution loop — your code or the LLM?
How that single choice shapes retry logic, failure recovery,
debuggability, cost, and production readiness.*

Douglas Mun

11 April 2026

Table of Content

Table of Content.....	2
Introduction.....	3
Two Approaches to Agent Development.....	3
Local Agent Frameworks.....	3
Managed Agent Platforms.....	3
The Core Difference: Who Is the Loop?.....	4
Architectural Trade-Offs.....	4
Retry on Transient Errors.....	4
Prompt Caching and System Prompt Hygiene.....	4
Context Filtering: Code vs Judgment.....	5
Failure Recovery: Exception Handler vs Reasoning.....	5
Full Trade-Off Table.....	6
Managed Platform Beta Realities.....	6
File Extraction Limitations.....	6
API Schema Inconsistencies.....	6
Sandbox Networking Reliability.....	7
Who Is the Loop — Across the Industry.....	7
LLM as the Loop.....	7
Claude Managed Agents.....	7
AutoGen / AG2 (Microsoft).....	7
Hybrid: Code Defines Structure, LLM Routes Within.....	8
CrewAI (Hierarchical Mode).....	8
OpenAI Agents SDK.....	8
Google ADK.....	8
Code as the Loop.....	8
LangGraph.....	8
Pydantic AI.....	8
Anthropic Claude SDK.....	8
Framework Comparison Summary.....	9
Key Takeaways.....	10
Decision Framework.....	11
Use a Local Framework When.....	11
Use a Managed Platform When.....	11
The Practical Recommendation.....	11
Sources.....	12
Anthropic Claude Managed Agents.....	12
Pydantic AI.....	12
Industry Frameworks.....	12

Introduction

The AI agent ecosystem in 2026 offers developers a fundamental architectural choice: build agents locally using open-source frameworks, or deploy them on a managed platform that handles orchestration, persistence, and infrastructure. This document compares these two approaches on their technical merits — architecture, cost, debuggability, failure handling, and production readiness — without tying the analysis to any specific project.

The comparison draws primarily on two representative technologies: **Pydantic AI** (the leading open-source, provider-agnostic Python agent framework) and **Anthropic Claude Managed Agents** (the first major cloud-hosted managed agent platform, launched in public beta on April 8, 2026). Where relevant, it broadens to cover the wider framework landscape: LangGraph, CrewAI, AutoGen/AG2, the OpenAI Agents SDK, Google ADK, and the Anthropic Claude SDK.

Two Approaches to Agent Development

Local Agent Frameworks

Local frameworks like Pydantic AI give you a library for *writing* agents. You define agents as Python classes, register tools with decorators, and orchestrate execution in your own code. The framework handles the LLM interaction protocol; you own everything else — retry logic, state management, session persistence, sandbox isolation, and deployment infrastructure.

The strengths are clear: full debuggability (Python stack traces), provider flexibility (swap between Anthropic, OpenAI, Gemini, or local models with one config change), complete testability (mock every external call), and zero vendor lock-in. The gap is equally clear: persistent sessions, managed sandboxing, credential scoping, context compaction, and automatic retry infrastructure are not included. You build that layer yourself.

Managed Agent Platforms

Managed platforms like Claude Managed Agents give you *infrastructure to run* agents. Anthropic’s engineering blog post on launch day describes the separation of three concerns that most frameworks conflate:

Component	Role	Why It’s Separate
Brain	Claude model — reasons, plans, calls, tools	Stateless; can be scaled, restarted, swapped
Hands	Execution sandbox — runs code, calls APIs	Isolated; credential-scoped by design
Session Log	Append-only event log outside the context window	Single source of truth; survives crashes

What this means for developers: persistent sessions, secure sandboxes, automatic context compaction, real-time SSE streaming, and multi-agent coordination — without writing infrastructure code. The trade-off: Anthropic-only, limited debuggability (session event log, no stack traces), and beta-era limitations on file extraction and sandbox networking.

The Core Difference: Who Is the Loop?

This is the most important concept in the entire comparison.

In a local framework, code is the loop. In a managed platform, the LLM is the loop.

In a local framework, your Python orchestrator is a deterministic state machine. It calls agents in a fixed sequence, collects results, and moves to the next stage. Every decision about what runs when, what gets retried, and what data flows where is written in code by you.

In a managed platform, the orchestrator agent (e.g. Claude Opus) reads the full situation — every tool result, every sub-agent output, every error — and decides what to do next through reasoning. The orchestration logic lives in the system prompt, not in code.

This single distinction cascades into every difference discussed below. It is not a subtle implementation detail — it is a fundamentally different model of what a “program” is.

Architectural Trade-Offs

Retry on Transient Errors

Rate limits are the most common real-world failure in multi-agent systems. Any agent calling external APIs (vulnerability databases, OSINT services, search engines) will eventually encounter 429 responses under heavy load.

Local framework: Without explicit retry code, a 429 propagates as an exception, aborting the affected stage. Downstream agents receive empty input. The scan produces zero findings — silently. The fix is a manual `_run_with_retry()` wrapper with exponential backoff, retryable substring matching, and an attempt ceiling. You write this yourself.

Managed platform: The API gateway absorbs transient 429 responses before they reach the orchestrator. If a sub-agent genuinely fails, the error appears in the orchestrator’s context, and it reasons about the next step: retry with modified instructions, fall back to alternative data sources, or continue with reduced confidence.

Key insight: the local framework retries the same call identically. The LLM orchestrator retries with different instructions based on what it learned from the failure. That gap compounds over the course of a long workflow.

Prompt Caching and System Prompt Hygiene

Anthropic’s prompt caching charges cache-hit tokens at 10% of the normal input price — a 90% cost saving on repeated context. For multi-agent systems where system prompts are 500–2,500 tokens each and run dozens of times per workflow, this is material.

The rule: system prompts must be static. Dynamic data (session IDs, input counts, timestamps) goes in the user message.

Local framework: This is a convention enforced by code comments. A future contributor can accidentally inject dynamic content into a system prompt, silently defeating caching on every call.

Managed platform: System prompts are baked into agent definitions at deploy time and stored server-side, referenced by agent ID. A developer *cannot* accidentally inject dynamic content. The architecture makes the correct behaviour the only possible behaviour.

Context Filtering: Code vs Judgement

As input size grows, the prompts sent to downstream agents become the bottleneck. Passing all upstream data uncritically risks hitting the context window limit and wastes tokens on irrelevant content.

Local framework: You write explicit filtering rules — e.g. only passing inputs that produced actionable results from earlier stages. This requires anticipating which data is relevant for each downstream agent and encoding that logic in code.

Managed platform: The LLM orchestrator reads upstream results with a genuine understanding and composes downstream prompts by hand, including only the relevant inputs because including empty ones would be pointless. The filtering is implicit in the reasoning, not explicit in code.

Failure Recovery: Exception Handler vs Reasoning

Local framework cascade: External API failure → stage aborted → downstream agents receive empty results → final report has no content → you don't know why unless you read the logs.

Managed platform cascade: External API failure → platform retries automatically → if still fails, sub-agent returns an error → orchestrator reads it in context → orchestrator reasons: "Data source is down, I'll instruct the agent to rely on alternative data and mark confidence lower" → workflow continues with degraded but real results.

The difference is not just recovery — it's informed recovery. The orchestrator produces a different output when a data source is unavailable than when it's available, because it understands the implication. A try/except block cannot do that.

Full Trade-Off Table

Dimension	Local Framework	Managed Platform
Retry logic	You write it (exponential backoff wrapper)	API gateway + LLM orchestrator reasoning
Prompt caching	Convention + code comments	Architecturally enforced at deploy
Context filtering	Explicit code rules per stage	LLM orchestrator judgement per spawn
Failure recovery	try/except + raise	Orchestrator reads error, adapts strategy
Parallelism	asyncio.gather() in code	Orchestrator spawns agents simultaneously
State management	Typed dataclass, explicit serialisation	Orchestrator's context window
Observability	Console output (manual wiring)	SSE event stream (built-in)
Cost	API calls only	API calls + session/agent overhead
Debuggability	Full Python stack traces	Session event log only
Determinism	High — same code = same run	Lower — LLM reasoning may vary
Provider flexibility	20+ providers, swappable per agent	Platform provider only
Test coverage	Full mock-based unit tests	Live sessions only (costs credits)
Offline / air-gap	Yes (with local model)	No — requires platform API

Managed Platform Beta Realities

Claude Managed Agents launched in public beta in April 2026. The following are platform-level observations that apply regardless of what you build on it.

File Extraction Limitations

The events list endpoint truncates `agent.tool_result` content at approximately **30KB**. Any file larger than this — databases, large JSON exports, binary artefacts — is effectively stuck in the sandbox. There is no upload endpoint, no file download endpoint, and no out-of-band transfer mechanism in the beta API. Workarounds involving base64 encoding inline only work for files under the truncation limit.

API Schema Inconsistencies

Pre-release documentation did not match the live API at launch. Seven field-level inconsistencies were found, each surfacing as a bare 400 Bad Request with no field-level detail. Examples:

What Documentation Suggested	What the API Requires
networking.type: "allowlist"	Only "limited" or "unrestricted"
environment_variables: {...}	The key is "environment"
python/system package keys	Must be pip / apt
max_tokens on agent definition	Not a valid field — remove it
web_search tool type	Not a valid tool type — remove it

Lesson: With any beta API, treat every field as unverified until you receive a 200. Budget 2–3 hours for schema debugging on first deployment.

Sandbox Networking Reliability

Sandbox networking appeared unreliable despite `networking.type: "unrestricted"` in the environment configuration. DNS resolution failed, TCP connect scans returned no results, and outbound connections timed out universally in some sessions.

The practical implication is that workflows requiring live network probing (port scanning, banner grabbing, TLS inspection) may produce degraded results with unverified confidence scores. The managed sandbox is currently better suited to analysis, correlation, and reporting than active network probing.

Who Is the Loop — Across the Industry

The “who is the loop” splits the entire agent framework landscape into three camps.

LLM as the Loop

The LLM decides what happens next. The developer defines agents and capabilities, but the execution path emerges from the model’s reasoning at runtime.

Claude Managed Agents

A single LLM orchestrator reasons across all agents, spawns sub-agents through judgment, and adapts strategy based on results. State lives in the orchestrator’s context window plus an append-only session log. Token efficiency is moderate — one orchestrator avoids multi-agent debate overhead. Debuggability is limited to the session event log. Platform-managed infrastructure provides persistent sessions and SSE streaming.

AutoGen / AG2 (Microsoft)

Multi-agent GroupChat in which an LLM selector selects the next speaker, and agents debate via dialogue. The purest “LLM as the loop” model among local frameworks — coordination emerges from conversation. The cost is high: each agent turn requires a full LLM call that includes the accumulated history. A 4-agent debate with 5 rounds requires a minimum of 20 LLM calls. Excels at quality-sensitive workflows where thoroughness matters more than speed. Model-agnostic with deep Azure optimisation.

Hybrid: Code Defines Structure, LLM Routes Within

The developer defines the topology in code, but an LLM makes routing decisions within that structure.

CrewAI (Hierarchical Mode)

Notable because it offers both philosophies as a configuration switch. In hierarchical mode, a manager LLM delegates to workers and validates quality. In sequential mode, it becomes pure “code as the loop.” Model-agnostic via LiteLLM. 43,000+ GitHub stars. The role-based mental model makes it the fastest path to a working prototype.

OpenAI Agents SDK

The core abstraction is the handoff: agents explicitly transfer control, carrying context. The LLM decides *when* and *to whom*, but handoff targets are predefined in code. High token efficiency (direct handoffs, no debate). Clean, opinionated API.

Google ADK

Hierarchical agent tree where a root delegates to sub-agents. Standout feature: native A2A (Agent-to-Agent) protocol support, enabling communication between agents from different frameworks. Gemini-optimised but supports other providers. The ecosystem is still maturing.

Code as the Loop

The developer defines the complete execution flow. The LLM sits within nodes and performs tasks, but code controls when tasks run and where data flows. Fully deterministic and predictable.

LangGraph

The low-level workhorse of the LangChain ecosystem. Directed cyclic graphs with nodes, edges, and explicit state transitions. An LLM can inform conditional edges, but the graph structure is fixed at compile time. Best debuggability in the ecosystem (LangSmith traces, time-travel debugging, checkpoint inspection). High token efficiency through direct state transitions. Model-agnostic. Used in production at Uber, JPMorgan, BlackRock, and Cisco.

Pydantic AI

The purest “code as the loop” model. Deterministic Python pipeline, every inter-stage dependency explicitly serialised. Zero orchestration overhead gives the highest token efficiency. Full stack traces and mockable architecture make it the most debuggable option—Model-agnostic across 20+ providers. V1 has been stable since September 2025. The trade-off: you own the entire operational layer.

Anthropic Claude SDK

Tool-use-first approach where agents are Claude models equipped with tools, including the ability to invoke other agents as tools. Minimal abstraction layer. Extended thinking (chain of thought visible in the API response), computer use, and MCP integration are the differentiators. Claude-only.

Framework Comparison Summary

Framework	Loop	Orchestration	State	Provider	Token Efficiency
Managed Agents	LLM	Single orchestrator reasons + spawns	Context window + session log	Anthropic	Moderate
AutoGen / AG2	LLM	Multi-agent GroupChat debate	Conversation history (in-memory)	Agnostic	Low (20+ calls)
CrewAI (Hierarchical)	Hybrid	Manager LLM delegates to workers	Task outputs + built-in state	Agnostic	Moderate
OpenAI Agents SDK	Hybrid	LLM-decided handoffs, fixed topology	Context variables (ephemeral)	OpenAI	High
Google ADK	Hybrid	Hierarchical tree, LLM routes within	Session state, pluggable backends	Gemini-0 pt.	Moderate
CrewAI (Sequential)	Code	Fixed task order, no LLM routing	Task outputs passed sequentially	Agnostic	High
LangGraph	Code	Directed graph, dev-defined transitions	Checkpointing + time-travel debug	Agnostic	High
Pydantic AI	Code	Linear pipeline, deterministic	Typed dataclass (explicit)	Agnostic	Highest
Claude SDK	Code	Tool-use loop, sub-agents as tools	Via MCP servers	Claude	High

Framework	Debuggability	Production Readiness	Best For
Managed Agents	Session event log only	Beta — managed infra, persistent sessions	Adaptive orchestration, analysis-heavy workloads
AutoGen / AG2	Difficult — async, distributed state	Production-ready, Microsoft-backed	Iterative reasoning, code gen with review loops
CrewAI (Hierarchical)	Moderate — role-based structure helps	Production-capable, AMP platform	Business workflows, rapid prototyping
OpenAI Agents SDK	Good — clean primitives, tracing	Production-grade, 19K+ stars	OpenAI-native apps, clean handoff patterns

Framework	Debuggability	Production Readiness	Best For
Google ADK	Vertex AI integration, maturing	Newer, stronger GCP integration	Google Cloud teams, multimodal, A2A protocol
CrewAI (Sequential)	Good — predictable, easy to trace	Production-capable	Simple pipelines, fastest prototyping path
LangGraph	Best — LangSmith, time-travel debug	Most mature: Uber, JPMorgan, Cisco	Complex stateful workflows, regulated industries
Pydantic AI	Best — full-stack traces, mockable	V1 stable; you own the ops layer	Dev/test, provider flexibility, air-gapped envs
Claude SDK	Good — extended thinking visible	Production-grade, MCP + computer use	Claude-native apps, tool-use-first architecture

Key Takeaways

LLM as the loop pays for adaptability with a token cost. AutoGen’s debate model consumes 20+ LLM calls per task; Managed Agents is leaner because the orchestrator reasons as a single agent rather than multiple agents debating.

Code as the loop wins in debuggability and efficiency, but requires anticipating every edge case up front. LangGraph leads in production maturity (checkpointing, time-travel debugging); Pydantic AI leads in testability (zero orchestration overhead).

Hybrid splits the difference — code defines structure, the LLM routes within it. CrewAI offers both modes as a config switch; the OpenAI Agents SDK achieves the cleanest hybrid design with its handoff abstraction.

The choice is not about which camp is “better” — it is about which trade-offs match your project. Many production systems use both a local framework for development and testing and a managed platform for production orchestration at scale.

Decision Framework

Use a Local Framework When

- **You need confirmed results from live probing** — active network scanning, API calls with full authentication, or workflows that require unrestricted outbound connectivity.
- **You are developing or debugging** — stack traces, mocks, and a fast test suite are worth more than any cloud feature during the build phase.
- **You need provider flexibility** — switching between Anthropic, OpenAI, and local models depending on cost, availability, or data residency.
- **Your output includes binary files**—databases, exports, or artefacts—that must be inspectable locally after the workflow completes.
- **You are running in an air-gapped environment** or in an environment without a reliable outbound internet connection to the platform API.

Use a Managed Platform When

- **You want LLM-driven cross-agent reasoning** — the intelligence fusion an LLM orchestrator performs (correlating discrepancies, discovering relationships, adapting strategy) is difficult to replicate in code.
- **Your workflow is primarily analysis and correlation** rather than live probing — where the orchestrator can reason across pre-fetched data without needing active network access.
- **You need persistent sessions** that survive client disconnects, with full event history retrievable after the fact.
- **You want zero infrastructure maintenance** — no retry code, no context management, no session state to manage.
- **You are running at scale**, where adaptive failure handling reduces wasted cycles more than it adds in overhead costs.

The Practical Recommendation

Develop and test with a local framework. Deploy and orchestrate with a managed platform. The two approaches are complementary, not competing. Share tools and agent logic between the two modes so that bugs caught locally never reach production.

As managed platforms mature — particularly around sandbox networking reliability and file extraction — the balance will shift further toward cloud-hosted operation. For now, local frameworks are the production workhorse, and managed platforms are the intelligent orchestration layer for analysis-heavy workloads.

Sources

Anthropic Claude Managed Agents

[Claude Managed Agents overview — Anthropic API Docs](#)

[Scaling Managed Agents: Decoupling the brain from the hands — Anthropic Engineering](#)

[Prompt caching — Anthropic API Docs](#)

Pydantic AI

[Pydantic AI v1: A Predictable & Robust GenAI Framework](#)

[Pydantic AI — Official Docs](#)

Industry Frameworks

[LangGraph — Agent Orchestration Framework](#)

[LangChain and LangGraph reach v1.0 milestones](#)

[Best Multi-Agent Frameworks in 2026 — GuruSup](#)

[Definitive Guide to Agentic Frameworks in 2026 — Softmax Data](#)